

AI-Driven Fire and Human Detection Model Using YOLOv9 with Automated Alert System

Mrs N. Vasavya, Assistant Professor

G. Iswarya, B. Jashwitha, J. Chandu, B. Sai Krishna

Department of Computer Science and Engineering (AI & ML)

Sri Venkateswara College of Engineering (Autonomous), Tirupati, India

Email: vasavya.n@svce.edu.in, 2022csm.r44@svce.edu.in, 2022csm.r17@svce.edu.in, 2022csm.r47@svce.edu.in, 2022csm.r22@svce.edu.in

Abstract—Real-time fire monitoring is a critical requirement in safety-sensitive environments such as homes, laboratories, warehouses, industries, academic campuses, office buildings, hospitals, and public infrastructure. Traditional fire alarm systems generally depend on smoke, temperature, or gas sensors and mainly provide threshold-based alerts without a visual understanding of the scene. Similarly, closed-circuit camera surveillance often requires continuous human observation, which becomes inefficient, error-prone, and difficult to scale when multiple camera feeds must be monitored simultaneously. To address these challenges, this paper presents an AI-driven fire and human detection framework using YOLOv9 with automated alerting and a web-based monitoring dashboard. The proposed system captures live video from a camera, preprocesses incoming frames, detects fire and human objects using a deep learning model, evaluates event severity through rule-based decision logic, and delivers alerts through email and SMS. A Flask-based dashboard provides live video streaming, fire and human counts, current system state, alerts, logs, and settings management. SQLite is used to store event records, user information, and configurable thresholds. The system distinguishes between fire-only and fire-with-human scenarios to improve situational awareness and support prioritised emergency response. The framework integrates object detection, web technologies, persistent logging, and notification services into a unified architecture suitable for intelligent surveillance and practical safety applications.

Index Terms—Fire Detection, Human Detection, YOLOv9, Deep Learning, Computer Vision, Flask, Real-Time Monitoring, Alert System, Surveillance, Safety Analytics

I. INTRODUCTION

Fire accidents are among the most dangerous emergencies in residential, industrial, educational, and public environments because they can escalate rapidly and cause severe damage to life, infrastructure, equipment, and operations. In many real-world situations, the severity of a fire-related incident depends heavily on response time. Even a small delay in detection or alert generation can allow flames to spread, smoke to accumulate, evacuation routes to become unsafe, and economic damage to increase significantly. For this reason, early detection and fast communication are essential requirements in modern safety systems.

Conventional fire safety mechanisms primarily rely on smoke sensors, heat detectors, thermal alarms, or gas sensors. These devices are important and widely used in practical deployment, but they mainly operate using fixed thresholds. As a result, they can trigger alarms without offering any scene-level interpretation. They do not provide information such as whether the flames are visually visible, how many hazardous

regions are present, whether a person is located near the fire, or whether the event appears isolated or critical. This limitation becomes significant when authorities or monitoring staff must verify the event before taking action.

Another common monitoring approach is the use of CCTV systems. While cameras provide rich visual information, manual monitoring of continuous video streams is labour-intensive and unreliable over long periods. Human operators may miss events because of fatigue, distraction, or simultaneous observation of multiple screens. Therefore, relying only on camera installation without intelligent automated analysis does not fully solve the problem of early hazard recognition.

Recent progress in deep learning and computer vision has enabled machines to automatically analyse images and video streams with increasing accuracy. Object detection models in particular have shown strong performance for locating and classifying objects in real time. Among them, the YOLO family has become especially popular because it combines fast inference with high practical usability in streaming environments [1]–[3]. YOLOv9 extends this family by introducing programmable gradient information and improved generalised efficient layer aggregation, which support stronger feature learning and better information propagation [4]. These characteristics make YOLOv9 suitable for real-time safety monitoring applications where both speed and detection reliability are important.

In this work, an AI-driven fire and human detection system is proposed using YOLOv9, OpenCV, Flask, SQLite, and automated notification services. The system continuously captures frames from a live camera, performs fire and human detection, applies event-based logic to classify the current state, displays the processed output on a web dashboard, and sends alerts when hazardous conditions are observed. Unlike a basic object detection script, the proposed framework is designed as a full monitoring application with live streaming, logs, thresholds, user management, and notification support.

The main contributions of this work are summarised below:

- Development of a real-time fire and human detection framework using YOLOv9.
- Event-aware status classification, such as Monitoring, Fire Detected, Human Detected, and Fire + Human Detected.
- Integration of a Flask-based web dashboard for live video, state visualisation, logs, and settings.

- Use of SQLite for persistent storage of logs, thresholds, settings, and user records.

The remainder of the paper is organised as follows. Section II reviews related work. Section III defines the problem. Section IV presents the objectives. Section V explains the proposed system architecture. Section VI describes the methodology and mathematical model. Section VII discusses the implementation. Section VIII presents results and discussion. Section IX outlines the advantages of the proposed system. Section X describes the limitations. Section XI presents applications. Section XII discusses future enhancements, and Section XIII concludes the paper.

II. RELATED WORK

Traditional fire monitoring systems have been built mainly on physical sensing devices such as smoke detectors, thermal detectors, and gas sensors. These systems are effective for basic fire safety and remain important in many buildings, but their operation is largely threshold-based. They do not analyse the visible scene and cannot provide contextual information regarding flame appearance, spread region, or nearby human presence.

Earlier vision-based fire detection techniques used hand-crafted image features such as colour distribution, flame motion, edge irregularity, texture variation, and temporal flicker analysis [10]. These methods aimed to identify visual fire patterns directly from frames, but they were often sensitive to illumination changes, bright light sources, reflections, and complex backgrounds.

Deep learning significantly improved the ability of machines to analyse images and extract discriminative features. Convolutional neural networks can automatically learn relevant spatial patterns from labelled examples and have shown strong performance in classification and detection tasks [9], [13]. Region-based object detectors provide high accuracy, but they are often computationally more expensive for streaming applications [15], [16]. In contrast, the YOLO family formulates detection as a single-stage problem, making it faster and more suitable for real-time deployment [1], [2]. YOLOv4 improved the balance between accuracy and speed [3], while YOLOv9 introduced stronger feature aggregation and gradient programming concepts that improve training and inference behaviour [4].

Human detection is also an essential component of intelligent surveillance systems. Earlier approaches, such as Histogram of Oriented Gradients, established a strong foundation for person detection [12], but modern deep learning models provide better robustness across scale variations, pose changes, and cluttered scenes. OpenCV is widely used for frame acquisition, annotation, preprocessing, and image encoding [5]. PyTorch provides an effective environment for loading trained models and running inference [7]. Flask is suitable for lightweight server-side integration, dashboard development, and API exposure in Python-based applications [8]. Although many studies focus on fire detection or surveillance individually, fewer works combine fire detection, human detection,

live dashboard monitoring, alert generation, access control, and database-backed persistence in one unified framework.

III. PROBLEM DEFINITION

Many existing monitoring systems still depend on manual observation or sensor-level alarms that provide limited contextual intelligence. This creates several practical limitations:

- Continuous CCTV observation requires permanent human attention.
- Traditional alarms do not visually describe the hazardous event.
- Most basic systems do not identify whether humans are present near the detected fire.
- Remote alerting through communication services may be absent.
- Logs, threshold control, and user management are often not integrated in a single platform.

A more effective safety monitoring system should continuously inspect live video, automatically detect visible fire, identify whether humans are present, classify the operational severity of the event, notify responsible users immediately, and allow event review through a web-based interface. Hence, there is a need for a real-time intelligent monitoring framework that combines object detection, alerting, visualisation, and storage into one practical system.

IV. OBJECTIVES

The main objectives of the proposed system are as follows:

- To develop a real-time fire and human detection system using deep learning.
- To detect fire and human presence simultaneously from live video streams.
- To design a web-based dashboard for real-time monitoring.
- To send automated alerts through email and SMS when a fire is detected.
- To maintain event logs and system data using a database.
- To improve safety monitoring using intelligent surveillance.
- To reduce response time by providing automated scene-aware alerts.
- To create a configurable and scalable monitoring solution for different deployment environments.

V. PROPOSED SYSTEM ARCHITECTURE

The proposed architecture integrates live video capture, frame preprocessing, deep learning inference, decision-level event analysis, alert services, dashboard rendering, and database persistence. The overall architecture is shown in Fig. 1.

The system contains the following functional modules.

A. Camera Acquisition Module

This module captures live frames from a webcam or camera source. It forms the input layer of the monitoring pipeline and continuously supplies image data for analysis.

In the implemented system, OpenCV is used for camera and frame capture. The video stream is accessed using the Video Capture function, which reads frames sequentially from the camera device.

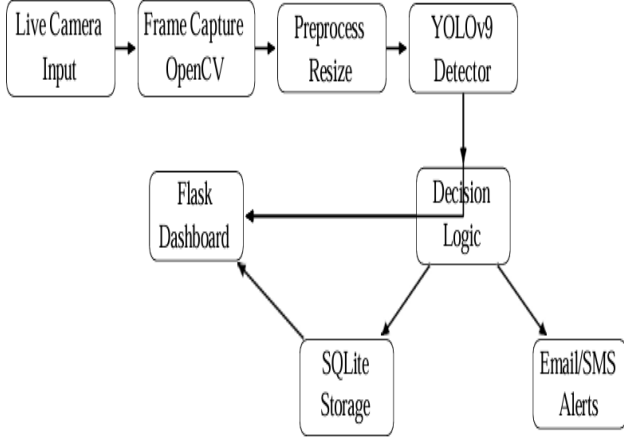


Fig. 1. Proposed system architecture for AI-driven fire and human detection.

B. Preprocessing Module

The incoming frames are resized and formatted for inference. Image preprocessing helps preserve stable model input dimensions and improves the consistency of real-time detection. Operations such as resizing, colour-space handling, normalisation, and frame buffering are performed before inference.

C. Detection Module

The YOLOv9 detector processes the preprocessed frame and predicts bounding boxes, class labels, and confidence scores for fire and person classes. This module performs the core visual intelligence of the system. The detector uses learned spatial features to localise hazardous regions and human presence in the monitored scene.

D. Decision Logic Module

The decision layer converts raw detections into meaningful monitoring states. It checks whether fire or humans are detected, evaluates the combination of classes, and determines the overall operational status. This module increases interpretability by converting detection outputs into actionable monitoring labels.

E. Alert Module

When a critical event is detected, the alert module sends email and SMS notifications to configured recipients. Cooldown logic is applied to avoid repeated alerts for the same event over consecutive frames. This reduces notification flooding and improves the practical usability of the system.

F. Dashboard Module

The Flask dashboard provides live processed video, fire and human counts, system status, navigation pages, and configuration options. It serves as the operational interface for users and administrators. The dashboard also supports logs, settings review, and state visualisation.

G. Database Module

SQLite is used to persist logs, users, thresholds, and settings. Persistent storage improves traceability and system control. Database support also helps in event history review, configuration management, and basic access control.

VI. METHODOLOGY AND MATHEMATICAL MODEL

The methodology includes dataset preparation, training, frame-level inference, class-wise counting, confidence filtering, state classification, and alert management.

A. Dataset Preparation

The fire detector is trained using labelled images arranged in YOLO format. The dataset is split into training, validation, and testing subsets. Each image is associated with a text annotation file containing normalised bounding box coordinates and class labels. A `data.yaml` file defines the number of classes, class names, and dataset paths.

Dataset diversity is an important requirement for fire detection. Flames may appear at different scales, colours, intensities, and shapes depending on indoor or outdoor conditions, object materials, and lighting. A robust dataset should therefore include varied scenes, backgrounds, distances, and camera angles to improve generalisation. A balanced dataset reduces overfitting and improves model stability under practical deployment conditions.

B. Model Training

YOLOv9 is selected because it supports real-time detection and improved feature learning [4]. During training, the model jointly learns localisation, classification, and objectness estimation. The total loss is defined as

$$L = L_{box} + L_{cls} + L_{obj} \quad (1)$$

where L_{box} is the bounding box regression loss, L_{cls} is the classification loss, and L_{obj} is the objectness loss.

The localisation term improves bounding box alignment, the classification term improves class discrimination, and the objectness term suppresses background noise. Joint optimisation of these terms is essential for stable object detection performance.

C. Frame-Level Inference

Let the live frame captured at time t be denoted by F_t . The detector output for that frame is represented as

$$D_t = \{(b_i, c_i, s_i)\}_{i=1}^n \quad (2)$$

where b_i denotes the predicted bounding box, c_i denotes the class label, and s_i denotes the confidence score of the i^{th} detection.

This representation indicates that every detected object in a frame is described by its location, category, and confidence. These outputs are later filtered and interpreted by the rule-based decision module.

D. Class-Wise Counting

The fire count is computed as

$$N_f = \sum_{i=1}^N I(c_i = \text{fire}) \quad (3)$$

and the human count is computed as

$$N_h = \sum_{i=1}^N I(c_i = \text{person}) \quad (4)$$

where $I(\cdot)$ is an indicator function that returns 1 when the given condition is true and 0 otherwise.

These counts are used as the primary quantitative indicators for state generation, dashboard display, and alert triggering.

E. Confidence Thresholding

To reduce low-confidence predictions, class-wise threshold filtering is applied:

$$\hat{D}_t = \{(b_i, c_i, s_i) \in D_t \mid s_i \geq \tau_{c_i}\} \quad (5)$$

where τ_{c_i} is the threshold selected for class c_i .

Confidence thresholding helps remove weak detections and improves prediction reliability. Different thresholds can be used for fire and human classes because the visual patterns and dataset difficulty levels may vary.

F. Decision Logic

The operational state is derived from the class-wise counts:

- if camera is unavailable, state = Camera Off;
- if $N_f > 0$ and $N_h > 0$, state = Fire + Human Detected;
- if $N_f > 0$ and $N_h = 0$, state = Fire Detected;
- if $N_f = 0$ and $N_h > 0$, state = Human Detected;
- otherwise, state = Monitoring.

This rule-based state model is simple, interpretable, and operationally useful. It converts low-level object detections into application-level status labels that are easier for users to understand.

G. Alert Cooldown

To prevent repeated alerts for the same continuing event, cooldown logic is introduced:

$$t - t_k \geq T_c \quad (6)$$

where t_k is the time of the last alert, and T_c is the cooldown interval.

This mechanism ensures that notifications are sent only after a sufficient interval, which reduces redundancy and prevents repeated disturbance during prolonged fire events.

VII. SYSTEM IMPLEMENTATION

A. Software Stack

The proposed system is implemented in Python because it provides strong support for deep learning, computer vision, and web services. OpenCV is used for frame capture, image preprocessing, and drawing annotations [5]. PyTorch provides the runtime environment for the trained detector [7]. YOLOv9 performs the object detection task [4]. Flask is used to build the web dashboard and API routes [8]. SQLite is selected as the database because it is lightweight, portable, and easy to integrate into academic prototype applications.

B. Backend Components

The backend is organised into multiple functional services:

- Detection service: handles model initialisation and frame processing.
- Streaming service: publishes the latest annotated frame through MJPEG.
- Status API: returns fire count, human count, and current state in JSON form.
- Logging service: stores critical events and alert activities.
- Settings service: manages thresholds, toggles, and alert targets.
- Authentication service: supports admin users and controlled dashboard access.

C. Dashboard Structure

The dashboard provides live video, fire count, human count, system status, alerts, logs, and settings. The workflow of the monitoring system is shown in Fig. 2, and the dashboard layout is shown in Fig. 3.

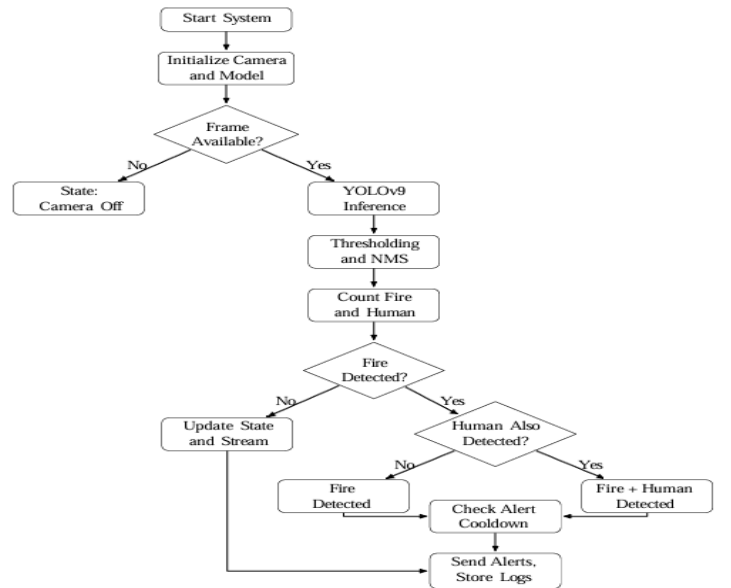


Fig. 2. Operational workflow of the proposed system.

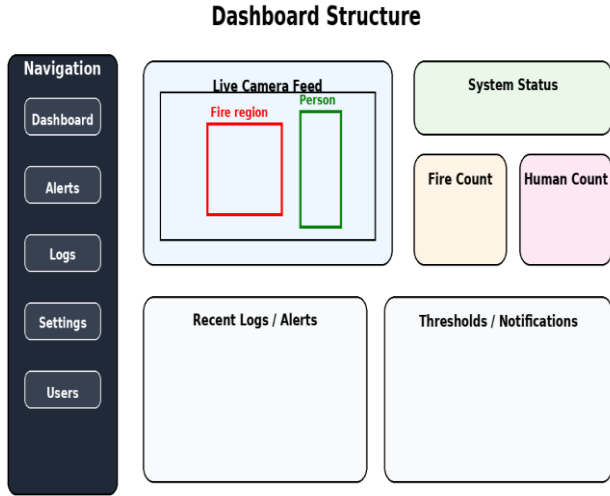


Fig. 3. Dashboard structure showing live feed, counts, alerts, logs, and settings.

TABLE I
RUNTIME PARAMETERS

Parameter	Value / Role
Input size	640 × 640
Fire threshold	Configurable
Human threshold	Configurable
IoU threshold	≈ 0.45
Alert cooldown	Configurable
Database	SQLite
Framework	Flask
Detector	YOLOv9

D. Algorithm

The runtime behaviour of the system can be summarised as follows:

- 1) Initialise the camera, detector, database, and notification service.
- 2) Capture a frame from the camera.
- 3) If the frame is unavailable, set the state to Camera Off.
- 4) Otherwise, preprocess the frame.
- 5) Run YOLOv9 inference.
- 6) Apply thresholding and non-maximum suppression.
- 7) Count fire and human detections.
- 8) Determine the system state.
- 9) Annotate the frame with labels and boxes.
- 10) Update dashboard frame buffer.
- 11) Save the event log if required.
- 12) If fire is detected and the cooldown is satisfied, send email alert and SMS alert.
- 13) Continue monitoring until the system is stopped.

VIII. RESULTS AND DISCUSSION

The system is evaluated using live camera input under multiple monitoring scenarios. The purpose of the evaluation is not limited to model prediction alone, but includes the end-to-end behavior of frame capture, inference, dashboard updates, event logging, and notification delivery.

The main scenarios considered are:

- No fire and no humans in the scene.
- Only a human is present in the scene.
- Only fire is visible in the scene.
- Fire and humans are visible simultaneously.
- Camera unavailable or failure state.

The proposed system successfully performs real-time detection and monitoring in practical testing conditions. Fire and human objects are detected from live video, and the processed frame is displayed in the dashboard together with class-wise counts and current status. This makes the monitoring output easier to interpret than raw inference results alone.

The decision logic plays a valuable role in transforming predictions into operational states. For example, a Fire Detected condition indicates hazard presence, while Fire + Human Detected suggests a more urgent event because a person is present in the affected scene. The Human Detected state confirms that the monitoring pipeline is functioning even in non-hazard scenarios. Such state-based interpretation improves usability and reduces operator workload.

The alerting subsystem improves practical usability by notifying responsible users through email and SMS. The cooldown mechanism is especially important because real fires may remain visible for many consecutive frames, and without cooldown, the same event could generate excessive repeated notifications.

The observed results indicate that the proposed framework is effective as an academic prototype for smart surveillance. Its performance depends on model quality, lighting conditions, camera position, and the complexity of the monitored environment. In controlled scenes, the system provides stable outputs and meaningful monitoring support.

TABLE II
SYSTEM COMPARISON

Feature	Traditional	Proposed
Detection basis	Smoke/heat	Vision-based DL
Human awareness	No	Yes
Scene context	Limited	Higher
Monitoring	Manual/basic	Web dashboard
Alerts	Alarm only	Email + SMS
Logs	Minimal	Stored in DB
Configurability	Low	Higher

IX. ADVANTAGES OF THE PROPOSED SYSTEM

The proposed system provides several advantages over traditional fire monitoring methods:

- Real-time visual fire detection using deep learning.
- Detection of both fire and human presence.
- Live monitoring through a web-based dashboard.

- Automated email and SMS alerts.
- Database-backed logging and configuration storage.
- Reduced need for continuous manual CCTV observation.
- Configurable thresholds for flexible deployment.
- Unified integration of detection, monitoring, alerting, and storage.

In addition to these benefits, the proposed framework increases scene awareness by combining visual detection with contextual decision logic. It also improves traceability through persistent logs and supports administrative control through settings and user management. These features make the system more practical than a standalone detection script.

X. LIMITATIONS

Despite its strengths, the system still has certain limitations:

- Bright fire-like regions, reflections, or strong lighting sources may occasionally generate false positives.
- Poor illumination, motion blur, camera noise, and low-resolution frames can reduce detection quality.
- CPU-only execution may reduce frame processing speed when compared with GPU-based deployment.
- Smoke-only events are not explicitly recognised unless smoke is included as a trained class.
- Performance depends strongly on dataset quality, annotation accuracy, and scene diversity.
- Occlusion or partial visibility of flames may reduce model confidence.
- Single-camera monitoring limits spatial coverage in large environments.
- Internet connectivity may be required for real-time email and SMS alert transmission.

These limitations show that practical deployment requires careful threshold tuning, better datasets, stable hardware, and suitable environmental conditions.

XI. APPLICATIONS

The proposed framework can be applied in homes, apartments, offices, laboratories, industrial plants, warehouses, server rooms, hospitals, and educational institutions. It is especially useful in environments where cameras are already installed, but intelligent event recognition is missing.

The system is also suitable for small-scale industrial safety monitoring, campus surveillance, smart building supervision, laboratory hazard monitoring, and centralised safety dashboards in restricted areas.

XII. FUTURE ENHANCEMENTS

The present framework can be extended further in several directions:

- Add additional hazard classes such as smoke, gas leaks, sparks, and suspicious objects.
- Support multiple camera streams for large-scale and multi-location monitoring.
- Introduce cloud deployment for centralised monitoring, remote access, and scalable storage.

- Apply temporal smoothing and video-based sequence analysis to reduce frame-level instability.
- Integrate IoT devices such as alarms, sprinklers, and emergency shutdown systems.
- Improve model efficiency through pruning, quantisation, or lightweight deployment models.
- Add mobile application support for remote alert viewing and system control.
- Introduce role-based access control with stronger authentication and audit tracking.
- Use GPS tagging or zone-based localisation for large industrial environments.
- Include performance evaluation metrics such as precision, recall, F1-score, and mAP in future experimental studies.

These enhancements can transform the current framework from an academic prototype into a more robust, real-world intelligent safety monitoring platform.

XIII. CONCLUSION

This paper presented an AI-driven fire and human detection system using YOLOv9 with automated alerting and a web-based monitoring dashboard. The proposed framework integrates live camera capture, deep learning-based object detection, class-wise event interpretation, web visualisation, alert communication, and database persistence into a unified safety monitoring solution. By detecting both fire and human presence, the system provides better situational understanding than traditional threshold-based alarms. The dashboard improves usability, while automated notifications improve response readiness. Overall, the work demonstrates that modern object detection models can be combined effectively with lightweight web technologies to build practical intelligent surveillance systems for safety-critical environments.

REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2016, pp. 779–788.
- [2] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2017, pp. 7263–7271.
- [3] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.
- [4] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," *arXiv preprint arXiv:2402.13616*, 2024.
- [5] G. Bradski, "The OpenCV library," *Dr Dobb's Journal of Software Tools*, 2000.
- [6] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2016.
- [7] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [8] M. Grinberg, *Flask Web Development*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.
- [9] D. Cireşan, U. Meier, and J. Schmidhuber, "Multi-column deep neural networks for image classification," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2012, pp. 3642–3649.
- [10] T. Celik and H. Demirel, "Fire detection in video sequences using a generic colour model," *Fire Safety Journal*, vol. 44, no. 2, pp. 147–158, 2009.

- [11] P. Barmpoutis, T. Papaioannou, K. Dimitropoulos, and N. Grammalidis, "A review on early forest fire detection systems using optical remote sensing," *Sensors*, vol. 20, no. 22, pp. 1–29, 2020.
- [12] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2005, pp. 886–893.
- [13] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Cham, Switzerland: Springer, 2022.
- [14] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.
- [15] R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Computer Vision*, 2015, pp. 1440–1448.
- [16] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017.
- [17] W. Liu et al., "SSD: Single shot multibox detector," in *Proc. European Conf. Computer Vision*, 2016, pp. 21–37.